NITTE (Deemed to be University) | NITTE MEENAKSHI INSTITUTE OF TECHNOLOGY

Department of Information Science and Engineering (ISE)

B. Tech 2nd Year, NDU Batch

Data Structures (25CSE201)

Under the guidance of

Assistant Professor Mrs. M. Lakshmi

Project on:

Smart Library Book Inventory & Due-Date Tracker

Submitted By:

NAME	USN
1. Sridhar Sahu	NB25ISE236
2. Sri Durga Siva Sai	NB25ISE234
3. Shrikar MJ	NB25ISE220
4. Shreesha K	NB25ISE215

Abstract

The Smart Library Book Inventory & Due-Date Tracker is a self-contained, high-performance desktop application built entirely in pure C to automate and streamline departmental library management. It combines a Binary Search Tree for rapid alphabetical catalogue lookups and a Min-Heap for real-time chronological due-date prioritization, eliminating the inefficiencies of manual spreadsheet tracking. Featuring a hardware-accelerated, immediate-mode graphical user interface powered by Raylib with dual light and dark academic themes, the application allows users to add books, perform smart substring searches, track return statuses, and ensure complete data safety through robust offline file I/O persistence.

The Smart Library Book Inventory & Due-Date Tracker is a high-performance desktop application designed to streamline library management by combining rigorous systems-level data structures with a modern graphical interface. Functionally, the application allows users to seamlessly add new book records, search the catalogue using smart case-insensitive substring matching, and track item return statuses in real time. Under the hood, it organizes the library catalogue alphabetically using a Binary Search Tree for rapid lookups and maintains a chronological priority queue via a Min-Heap to instantly flag overdue returns without manual sorting. Furthermore, it ensures complete offline data safety by automatically saving and loading records from local disk storage, while presenting the entire workflow through a sleek, hardware-accelerated desktop window featuring customizable light and dark academic themes.

Introduction

Modern institutional, campus, and departmental libraries frequently encounter significant operational bottlenecks driven by legacy tracking methods. Traditional workflows often rely heavily on flat spreadsheet applications or bloated, resource-heavy desktop software solutions. While spreadsheets offer flexibility, they entirely lack automated priority queuing, real-time date evaluation, and standalone executable portability. Conversely, enterprise-grade web or heavy GUI frameworks introduce excessive runtime overhead, complex dependency chains, and unnecessary server hosting requirements for local campus environments. Consequently, manual due-date auditing frequently leads to administrative friction, unmanaged book losses, and delayed priority alerts for overdue items.

To overcome these structural limitations, this project introduces a high-performance, resource-efficient desktop library management system engineered entirely in pure C. By leveraging a systems-level programming language, the application achieves near-instant execution speeds, an exceptionally low memory footprint, and complete independence from external runtime environments. At the core of the software is a robust hybrid data structure design tailored specifically to handle inventory management and chronological urgency simultaneously.

The catalogue indexing layer is powered by a custom Binary Search Tree (BST). The BST organizes the entire library inventory alphabetically by book title, ensuring logarithmic time complexity ($O(\log n)$) for insertions, deletions, and structural lookups. This alphabetical ordering allows administrators to scan and manage catalogue listings smoothly. Operating in tandem with the BST is a chronological priority engine built on a Min-Heap. The Min-Heap orders books based on their due dates, which are stored as standardized integer values (YYYYMMDD). By maintaining the earliest due date at the root node, the Min-Heap provides constant-time ($O(1)$) peak access to the most critical or overdue books, allowing library staff to immediately identify and address overdue materials without scanning the entire database.

To ensure data integrity and prevent information loss across application restarts, the system incorporates a robust file I/O persistence layer. Whenever modifications occur—such as adding new inventory items or updating return statuses—the application can serialize its runtime state into a structured local snapshot file (library_data.txt). Upon subsequent launches, the persistence engine deserializes this file, rebuilding both the alphabetical BST catalogue and the chronological Min-Heap priority queue instantaneously to restore the full operational state.

User interaction is handled through a modern, hardware-accelerated, immediate-mode graphical user interface built using Raylib and Raygui. Operating at a locked and stable 60 FPS, the UI features a clean split-pane dashboard that separates inventory input forms from active data grids. To accommodate different user preferences and study environments, the system incorporates a dual-mode academic visual theme (*Paper & Sage* palette) supporting seamless transitions between a crisp light mode and a cozy dark mode. Furthermore, readability is optimized through custom text rendering, balanced padding, and high-contrast typography.

The application also includes several advanced developer features designed to enhance usability. A smart, case-insensitive substring search engine allows users to query partial keywords—such as typing "clean"—to dynamically surface matching records like "Clean Code" across the catalogue. Additionally, a real-time "Mark as Returned" workflow enables users to toggle book availability instantly. When a book is marked as returned, its status shifts dynamically from overdue or active to available, and it is automatically bypassed in subsequent Min-Heap priority evaluations.

By merging rigorous systems-level data structure design with a clean, hardware-accelerated graphical interface, this project delivers an optimal, self-contained solution for local library automation, striking an ideal balance between computational efficiency and modern user experience.

Problem Statement

Traditional departmental libraries rely on passive spreadsheets or bloated enterprise software, leading to administrative overhead and unmanaged book losses. This project addresses the lack of a fast, resource-efficient desktop tool by delivering a self-contained C application that automates catalogue indexing and real-time overdue auditing.

Objectives

Objective 1: Algorithmic Catalogue Indexing and Advanced Substring Search

- **Focus Area:** The Binary Search Tree (BST) backend engine (bst.c) and search routing.
- **Elaboration:** This objective establishes an automated, case-insensitive alphabetical catalogue structure using a Binary Search Tree, ensuring that all primary catalogue operations—including insertions, deletions, and structural lookups—achieve logarithmic time complexity ($O(\log n)$). Furthermore, it integrates a smart substring search engine that recursively traverses subtrees via strstr(), allowing users to query partial keywords (such as typing "clean" to dynamically surface "Clean Code" across the catalogue) without relying on external database indexes.

Objective 2: Real-Time Priority Queuing and Dynamic Item Lifecycle Management

- **Focus Area:** The Min-Heap priority queue module (heap.c) and status controllers.
- **Elaboration:** This objective addresses temporal urgency and automated overdue auditing by implementing a Min-Heap data structure ordered by standardized due-date integers (YYYYMMDD), granting instant constant-time ($O(1)$) peak access to the most critical overdue item. Coupled with this is a real-time "Mark as Returned" workflow: when a user toggles a book as returned, its status shifts dynamically from active/overdue to available, and the system automatically bypasses it in subsequent Min-Heap priority evaluations to maintain precise inventory tracking.

Objective 3: Zero-Dependency Offline File I/O State Persistence

- **Focus Area:** The disk serialization and file management layer (persistence.c).
- **Elaboration:** These objective guarantees system reliability and data portability without depending on external database daemons or cloud servers. The persistence engine serializes the live runtime state of both the alphabetical BST catalog and the Min-Heap priority queue into a structured local text snapshot (library_data.txt). Conversely, upon application startup, the engine safely parses this file to reconstruct complete in-memory data structures, ensuring all inventory additions, modifications, and return statuses survive application closures seamlessly.

Objective 4: Hardware-Accelerated Immediate-Mode Graphical User Interface

- **Focus Area:** The Raylib and Raygui graphics rendering pipeline (main.c).
- **Elaboration:** This objective delivers a responsive, lightweight, and modern desktop user experience operating at a stable, hardware-accelerated 60 FPS without the heavy resource consumption associated with web-based frameworks or enterprise desktop toolkits. It includes the implementation of a dual-mode academic visual theme (*Paper & Sage* palette) supporting smooth transitions between a crisp light mode and a cozy dark mode, accompanied by careful input padding and typography scaling to maximize layout readability.

Project Scope & Relevance

The project scope encompasses the design, development, and deployment of a self-contained, high-performance desktop application tailored specifically for institutional, campus, and departmental libraries. Functionally, the application covers core library operations including alphabetical book cataloguing, automated chronological due-date prioritization, smart case-insensitive substring searching, real-time return status toggling, and robust offline file I/O data persistence. Technically, the scope is strictly defined around a pure C implementation utilizing a Binary Search Tree (BST) for inventory indexing, a Min-Heap for priority queue evaluation, and a hardware-accelerated immediate-mode graphical user interface built with Raylib and Raygui running at a stable 60 FPS. It deliberately excludes cloud-hosted databases, multi-client server infrastructures, and heavy web frameworks, focusing entirely on a resource-efficient, offline-first desktop tool. The relevance of this project lies in its ability to bridge a distinct technological gap between passive flat spreadsheets and bloated enterprise software for smaller-scale library environments. Many departmental and campus libraries struggle with administrative overhead because traditional tools lack automated priority queuing and real-time date auditing. By implementing rigorous data structures—specifically a BST for logarithmic search efficiency ($O(\log n)$) and a Min-Heap for constant-time ($O(1)$) peak access to overdue items—within a systems-level programming language, the project achieves near-instant execution speeds and a minimal memory footprint. Furthermore, its dual-mode academic visual theme, zero-dependency architecture, and practical real-world utility make it exceptionally relevant for institutional automation, resource-constrained environment, and academic software engineering evaluation.

Gap Identification

Institutional, campus, and departmental libraries face a distinct technological gap because existing solutions are divided between passive flat-file spreadsheets that lack automated priority queuing and standalone portability, and bloated enterprise systems that introduce excessive runtime overhead and complex database dependencies. This project bridges that divide by delivering a self-contained desktop application that combines a Binary Search Tree for alphabetical catalogue lookups and a Min-Heap for constant-time priority evaluation with a zero-dependency, hardware-accelerated graphical user interface, thereby automating due-date auditing and offline persistence without computational bloat.

Justified Tech Stack Selection:

Programming Language: Pure C (C11 standard)

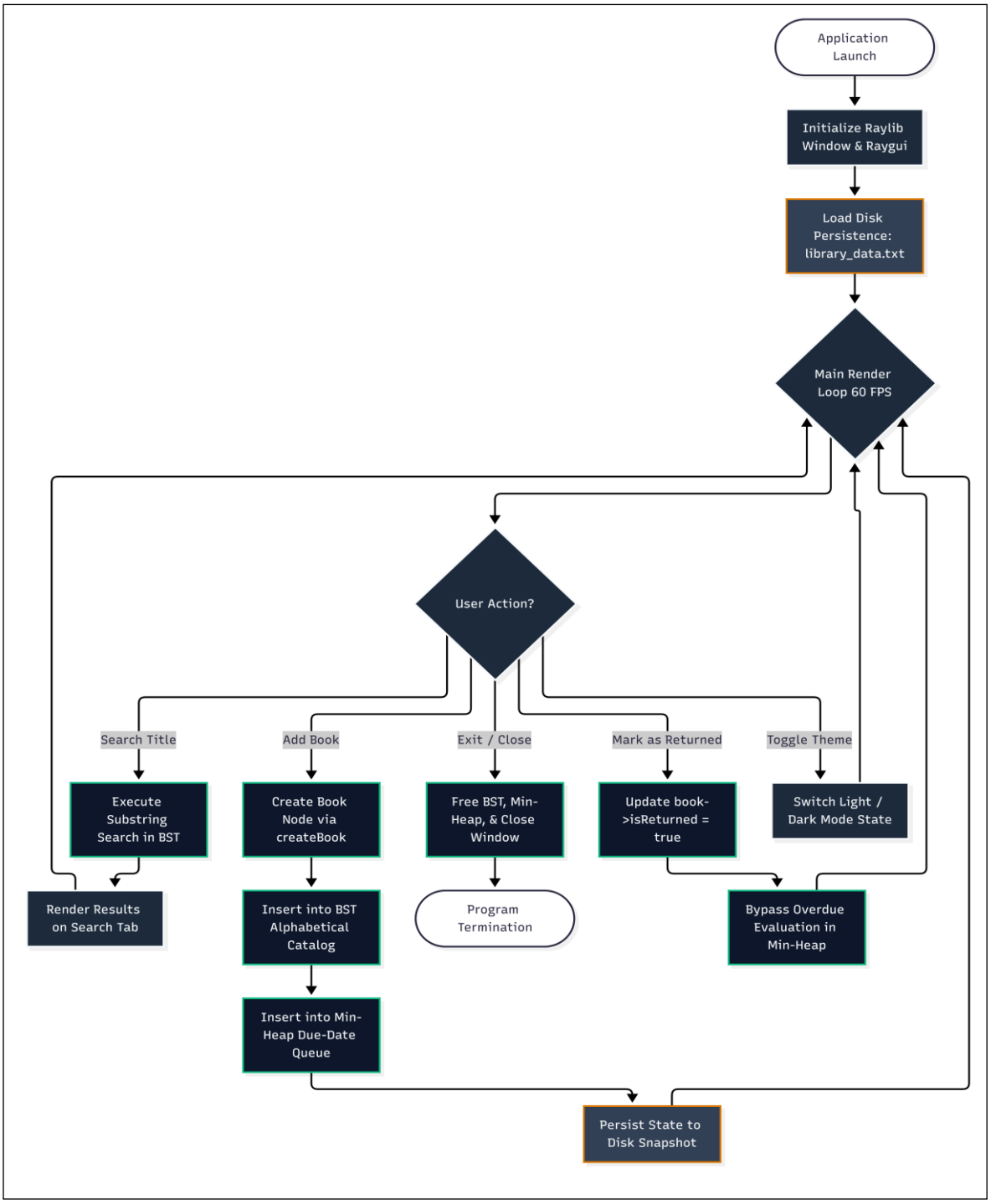
- **Functioning:** Serves as the backbone of the application. C provides absolute control over memory allocation, pointer manipulation, and data structure design. By avoiding garbage collection and heavyweight virtual machines, it guarantees near-instant execution speeds, minimal RAM consumption, and a lightweight standalone binary footprint.

Data Persistence Layer: Standard C File I/O (stdio.h)

- **Functioning:** Handles state serialization and deserialization by writing and reading from a structured local snapshot file (library_data.txt). This allows the system to save inventory modifications and restore complete in-memory data structures across application sessions without requiring external database daemons or cloud servers.

System Architecture and Design

Overall Workflow flowchart:

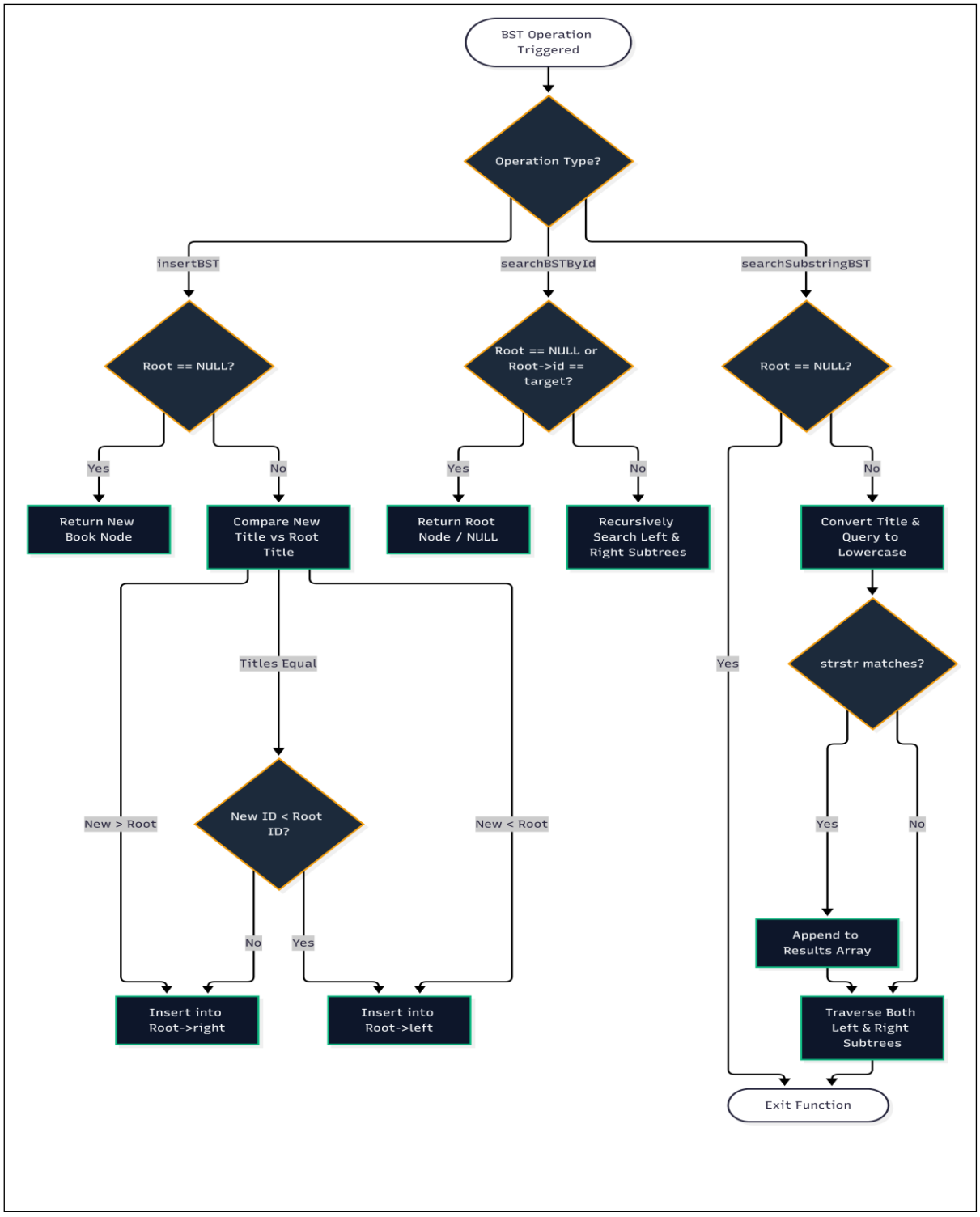


The overall workflow of the Smart Library Book Inventory & Due-Date Tracker follows a cyclical, modular pipeline that bridges user interaction, data processing, and persistent storage. Upon launching the application, the system initializes the hardware-accelerated Raylib graphics window and immediately triggers the persistence module to load saved inventory records from the local library_data.txt file, reconstructing both the alphabetical Binary Search Tree (BST) catalogue and the chronological Min-Heap priority queue in memory. Once initialized, the application enters a stable, high-performance 60 FPS render loop where it continuously clears the canvas, applies the active academic visual theme (Light or Dark mode), and renders the split-pane desktop dashboard containing inventory input forms and interactive table grids.

As the user interacts with the application—whether by submitting a new book entry, running a smart substring search, toggling a book's availability via the "Mark as Returned" button, or switching view tabs—the main controller captures these events and routes them to the appropriate underlying data engine. New book submissions are processed by the BST engine to ensure case-insensitive alphabetical sorting and are simultaneously indexed into the Min-Heap priority queue based on their standardized due-date integers. The Min-Heap constantly maintains the most critical overdue item at its root for instant $\boldsymbol{O(1)}$ access, automatically bypassing any books that have been flagged as returned. Finally, whenever significant structural changes occur or snapshots are manually triggered, the persistence engine serializes the live in-memory catalog back to disk, ensuring complete offline data safety before the user eventually closes the window and the system executes a clean memory deallocation across all modules.

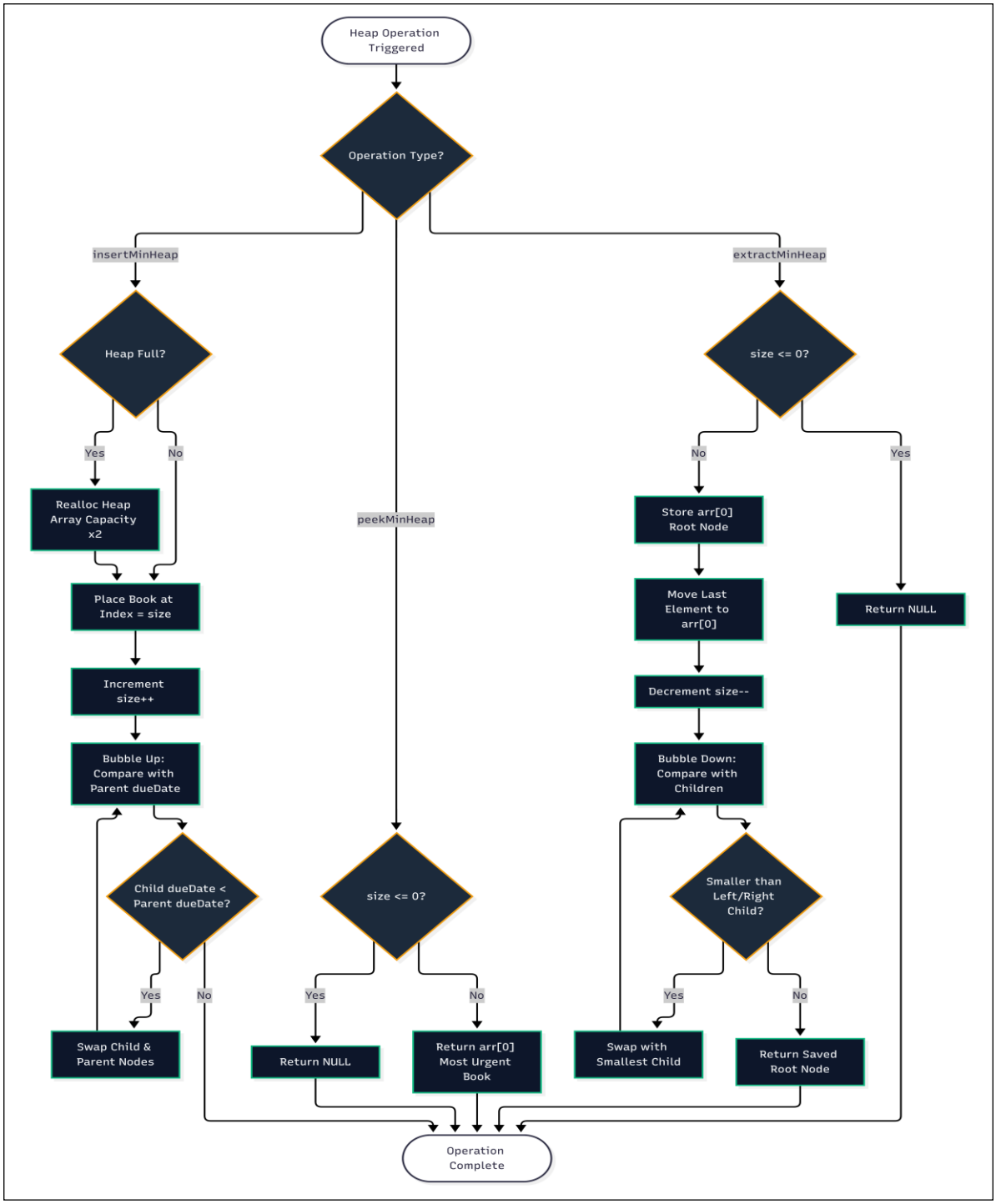
Modular Codebase & Data Flow Architecture:

BST Catalog Engine (bst.c & bst.h):



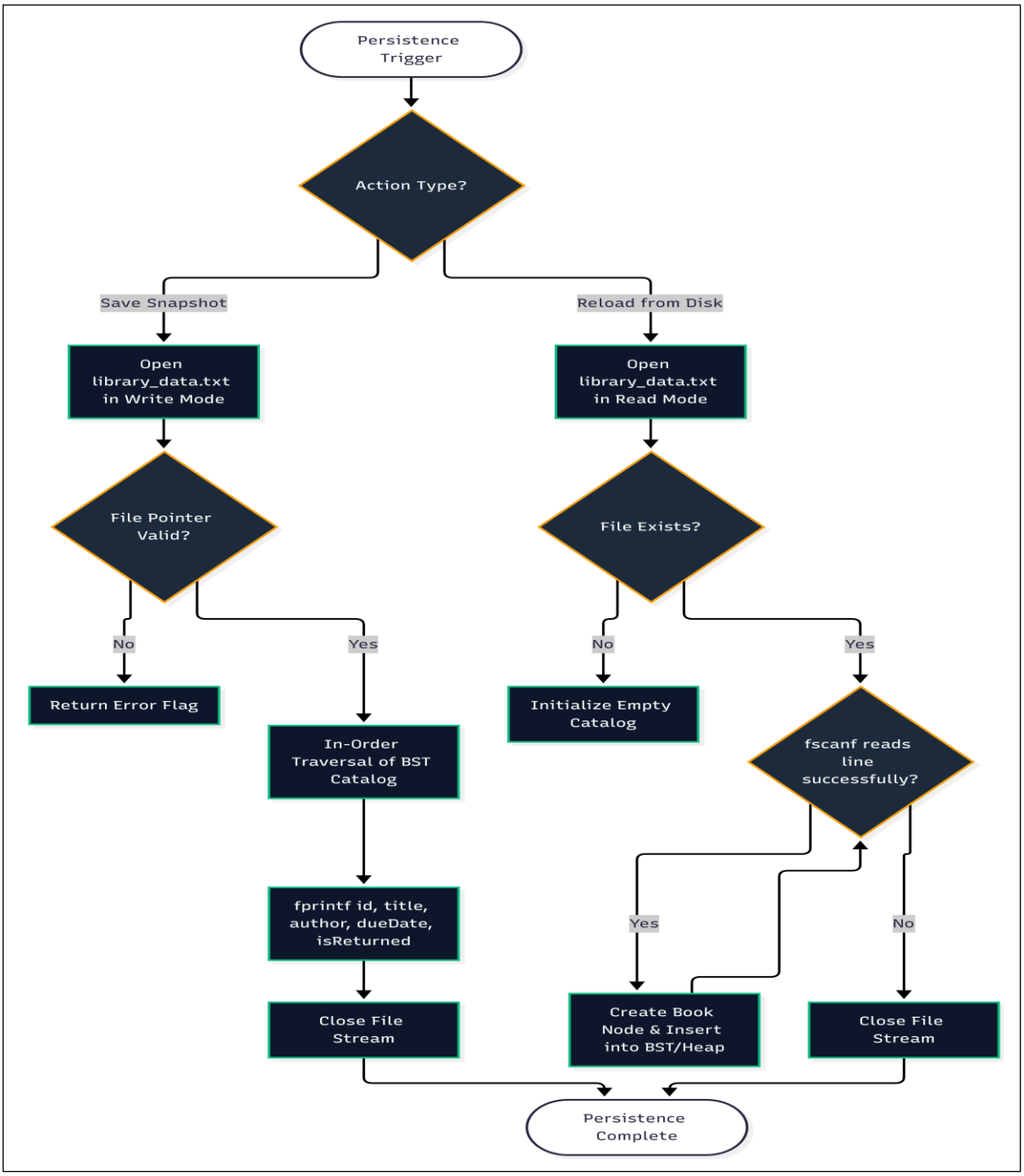
The Binary Search Tree catalogue engine acts as the primary indexing structure for the library inventory, organizing all book records alphabetically by their titles to ensure logarithmic time complexity ($O(\log n)$) for insertions, deletions, and structural lookups. By leveraging case-insensitive alphabetical sorting, the module maintains a clean structural order as the inventory expands, while also providing advanced capabilities like a smart substring search engine that scans subtrees recursively to match partial keywords.

Min-Heap Priority Queue Module (heap.c & heap.h):



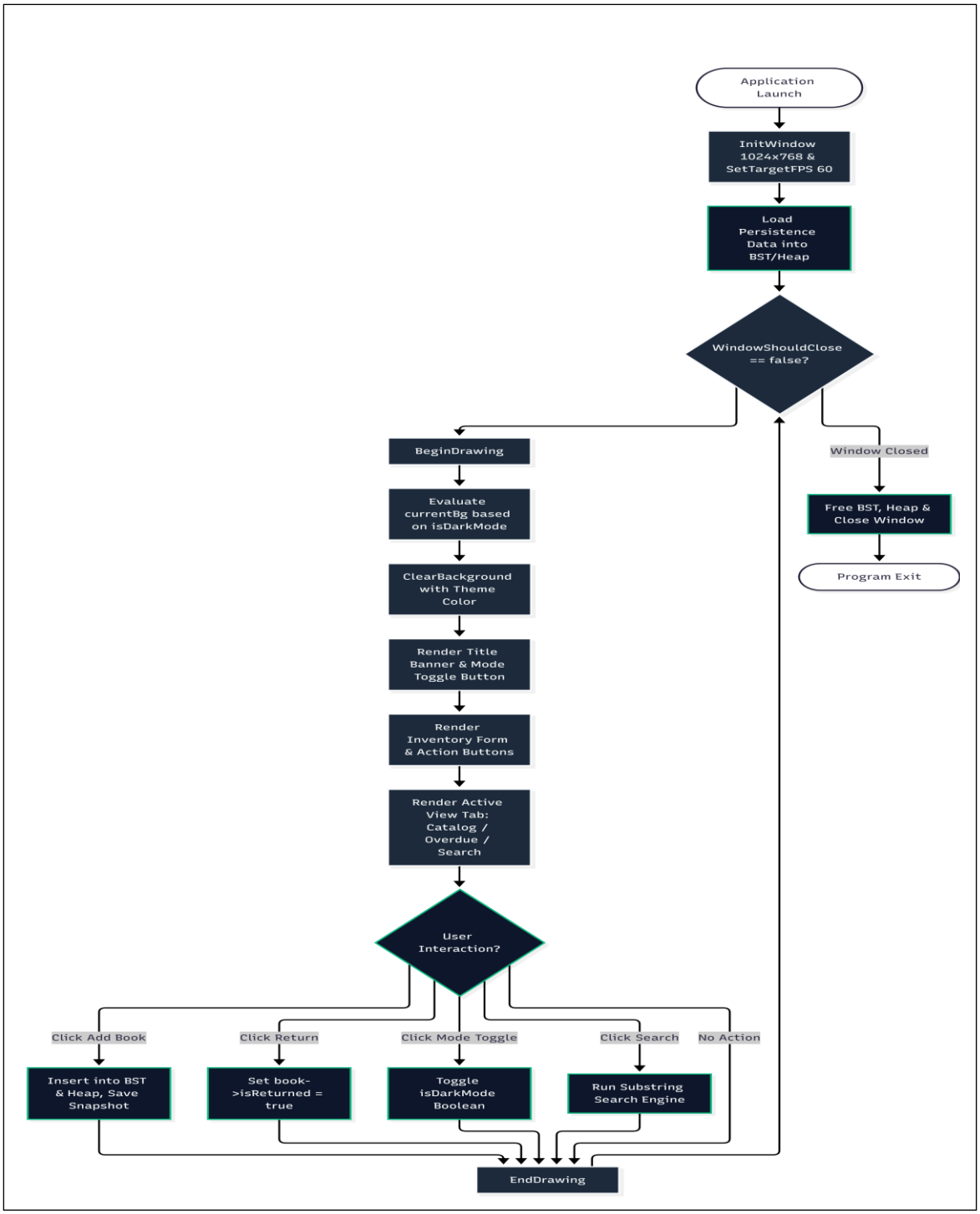
The Min-Heap priority queue module manages temporal urgency and overdue auditing by ordering book nodes chronologically based on their due dates, which are stored as standardized integers (YYYYMMDD). By maintaining the earliest due date at the root of the tree, this module provides constant-time ($O(1)$) peak access to the most critical overdue items, allowing administrators to instantly evaluate inventory urgency without scanning non-urgent records.

Persistence & File I/O Engine (persistence.c):



The persistence and file I/O layer guarantees system reliability and data portability by serializing the live runtime state of both the BST catalogue and the Min-Heap priority queue into a structured local text snapshot (library_data.txt). Upon application startup, this engine safely parses the file to reconstruct the complete in-memory data structures, ensuring that all inventory additions, modifications, and return statuses survive application closures seamlessly without requiring external database servers.

UI & Main Controller Module (main.c):



The user interface and main controller module manages the Raylib graphics rendering pipeline, event routing, and application state loop at a stable, hardware-accelerated 60 FPS. It provides a clean split-pane desktop dashboard featuring inventory input forms, interactive table grids, and a dual-mode academic visual theme (*Paper & Sage* palette) that supports smooth transitions between light and dark modes, alongside real-time state synchronization for features like the “Mark as Returned” workflow.

Initial Work Done:

Working Environment: Fully established C compilation pipeline using GCC and Raylib across Linux/Windows environments.

Core Modules Completed:

- Fully operational BST insertion, deletion, and smart case-insensitive substring search (e.g., matching "clean" to "Clean Code").
- Functional Min-Heap overdue evaluator comparing book due dates against a dynamic reference date.
- Implemented "Mark as Returned" workflow, updating book statuses (AVAILABLE / ACTIVE / OVERDUE) in real time.
- Styled GUI complete with Light and Dark academic themes (*Paper & Sage* palette), input padding, and tabular inventory grids.

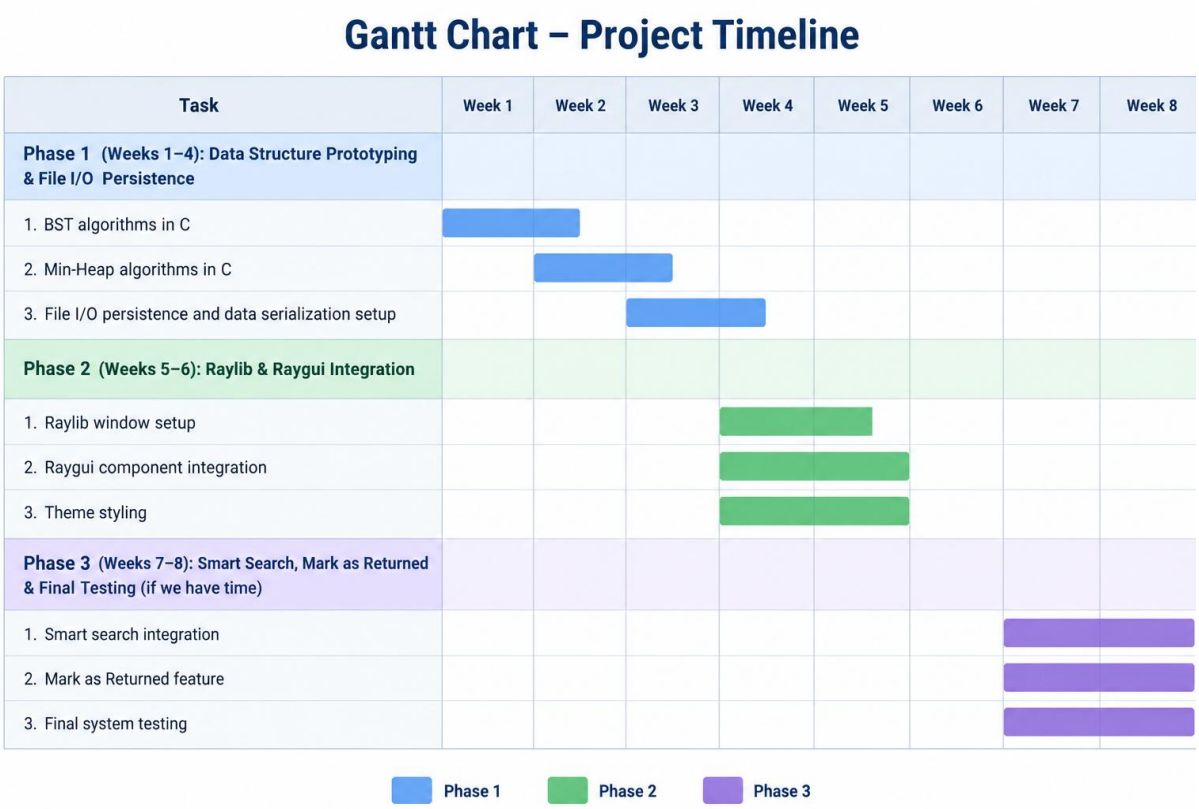
Planning, Presentation

Team Task Distribution (4-Member Matrix):

- **Member 1:** BST Catalogue & Substring Search Engine (bst.c)
- **Member 2:** Min-Heap Overdue & Priority Logic (heap.c)
- **Member 3:** File Persistence & State Controller (persistence.c)
- **Member 4:** Raylib/Raygui UI, Theme Management & Main Loop (main.c)

Project Timeline / Gantt Summary:

- *Phase 1 (Weeks 1–4):* Data structure prototyping (BST & Min-Heap algorithms in C) and, file I/O persistence and data serialization setup.
- *Phase 2 (Weeks 5–6):* Raylib window setup, Raygui component integration, and theme styling.
- *Phase 3(Weeks 7–8):* Smart search integration, "Mark as Returned" feature, and final system testing. (if we have time)



Work done until now:

1. Binary Search Tree (BST) Module

bst.h *(Header File)*

```
#ifndef BST_H
#define BST_H
#include <stdbool.h>
// Book node structure for BST and heap integration
typedef struct Book {
    int id;
    char title[100];
    char author[100];
    int dueDate;       // Stored as YYYYMMDD integer (e.g., 20260930)
    bool isReturned;   // false = Active/Checked Out, true = Returned
    struct Book *left;
    struct Book *right;
} Book;

// Function Prototypes
Book* createBook(int id, const char *title, const char *author, int dueDate);
Book* insertBST(Book *root, Book *newBook);
Book* searchBSTById(Book *root, int id);
void searchSubstringBST(Book *root, const char *query, Book *results[], int *resultCount,
int maxResults);
void freeBST(Book *root);
#endif
```

bst.c *(Implementation File)*

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "bst.h"
```

```
// Allocate and initialize a new book node
Book* createBook(int id, const char *title, const char *author, int dueDate) {
    Book *newNode = (Book*)malloc(sizeof(Book));
    if (!newNode) return NULL;

    newNode->id = id;
    strncpy(newNode->title, title, sizeof(newNode->title) - 1);
    newNode->title[sizeof(newNode->title) - 1] = '\0';

    strncpy(newNode->author, author, sizeof(newNode->author) - 1);
    newNode->author[sizeof(newNode->author) - 1] = '\0';

    newNode->dueDate = dueDate;
    newNode->isReturned = false;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

// Insert book into BST ordered alphabetically by title
Book* insertBST(Book *root, Book *newBook) {
    if (root == NULL) return newBook;

    int cmp = strcasecmp(newBook->title, root->title);
    if (cmp < 0) {
        root->left = insertBST(root->left, newBook);
    } else if (cmp > 0) {
        root->right = insertBST(root->right, newBook);
    } else {
        // If titles match, sort by ID
        if (newBook->id < root->id)
            root->left = insertBST(root->left, newBook);
        else
            root->right = insertBST(root->right, newBook);
    }
    return root;
```

```
}

// Search BST by Book ID
Book* searchBSTById(Book *root, int id) {
    if (root == NULL || root->id == id) return root;
    Book *found = searchBSTById(root->left, id);
    if (found != NULL) return found;
    return searchBSTById(root->right, id);
}

// Helper: Convert string to lowercase for case-insensitive substring search
static void toLowerStr(char *dest, const char *src) {
    int i = 0;
    while (src[i]) {
        dest[i] = tolower((unsigned char)src[i]);
        i++;
    }
    dest[i] = '\0';
}

// Smart Substring Search (Fuzzy/Keyword Match across BST)
void searchSubstringBST(Book *root, const char *query, Book *results[], int *resultCount,
int maxResults) {
    if (root == NULL || *resultCount >= maxResults) return;

    char titleLower[100], queryLower[100];
    toLowerStr(titleLower, root->title);
    toLowerStr(queryLower, query);

    if (strstr(titleLower, queryLower) != NULL) {
        results[*resultCount] = root;
        (*resultCount)++;
    }

    // Traverse both subtrees since alphabetical BST order doesn't restrict substring location
    searchSubstringBST(root->left, query, results, resultCount, maxResults);
    searchSubstringBST(root->right, query, results, resultCount, maxResults);
```

```
}

// Free all nodes in the BST
void freeBST(Book *root) {
    if (root == NULL) return;
    freeBST(root->left);
    freeBST(root->right);
    free(root);
}
```

2. Min-Heap Priority Queue Module

heap.h *(Header File)*

```
#ifndef HEAP_H
#define HEAP_H

#include "bst.h"

// Min-Heap structure storing pointers to Book nodes, ordered by dueDate (YYYYMMDD)
typedef struct {
    Book **arr;
    int capacity;
    int size;
} MinHeap;

// Function Prototypes
MinHeap* createMinHeap(int capacity);
void insertMinHeap(MinHeap *heap, Book *book);
Book* peekMinHeap(MinHeap *heap);
Book* extractMinHeap(MinHeap *heap);
void freeMinHeap(MinHeap *heap);

#endif
```

heap.c *(Implementation File)*

```
#include <stdio.h>
#include <stdlib.h>
#include "heap.h"

// Initialize Min-Heap with given capacity
MinHeap* createMinHeap(int capacity) {
    MinHeap *heap = (MinHeap*)malloc(sizeof(MinHeap));
    if (!heap) return NULL;

    heap->capacity = capacity;
    heap->size = 0;
    heap->arr = (Book**)malloc(capacity * sizeof(Book*));
    if (!heap->arr) {
        free(heap);
        return NULL;
    }
    return heap;
}

// Helper: Swap two book pointers in the heap array
static void swapBooks(Book **a, Book **b) {
    Book *temp = *a;
    *a = *b;
    *b = temp;
}

// Helper: Bubble up to maintain Min-Heap property based on dueDate
static void heapifyUp(MinHeap *heap, int idx) {
    int parent = (idx - 1) / 2;
    while (idx > 0 && heap->arr[idx]->dueDate < heap->arr[parent]->dueDate) {
        swapBooks(&heap->arr[idx], &heap->arr[parent]);
        idx = parent;
        parent = (idx - 1) / 2;
    }
}
```

```
// Helper: Bubble down to maintain Min-Heap property
static void heapifyDown(MinHeap *heap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < heap->size && heap->arr[left]->dueDate < heap->arr[smallest]->dueDate)
        smallest = left;

    if (right < heap->size && heap->arr[right]->dueDate < heap->arr[smallest]->dueDate)
        smallest = right;

    if (smallest != idx) {
        swapBooks(&heap->arr[idx], &heap->arr[smallest]);
        heapifyDown(heap, smallest);
    }
}

// Insert a book into the Min-Heap
void insertMinHeap(MinHeap *heap, Book *book) {
    if (heap->size >= heap->capacity) {
        // Dynamic resizing if capacity is reached
        heap->capacity *= 2;
        Book **temp = (Book**)realloc(heap->arr, heap->capacity * sizeof(Book*));
        if (!temp) return;
        heap->arr = temp;
    }

    int idx = heap->size;
    heap->arr[idx] = book;
    heap->size++;
    heapifyUp(heap, idx);
}

// O(1) Access to the most urgent/earliest due date book
Book* peekMinHeap(MinHeap *heap) {
```

```
    if (heap->size <= 0) return NULL;
    return heap->arr[0];
}

// Extract the root (most urgent due date) from the Min-Heap
Book* extractMinHeap(MinHeap *heap) {
    if (heap->size <= 0) return NULL;

    Book *root = heap->arr[0];
    heap->arr[0] = heap->arr[heap->size - 1];
    heap->size--;
    heapifyDown(heap, 0);

    return root;
}

// Free the Min-Heap structure (does not free underlying book nodes managed by BST)
void freeMinHeap(MinHeap *heap) {
    if (heap) {
        free(heap->arr);
        free(heap);
    }
}
```

3. UI code

1. Raygui Headers & Custom TrueType Font Renderer

```
#include "raylib.h"
#define RAYGUI_IMPLEMENTATION
#include "raygui.h"
// High-Legibility Modern TrueType Font Engine
static Font mainFont;
static bool fontLoaded = false;
// Universal DrawTextEx wrapper for crisp TrueType rendering
void DrawAppText(const char* text, int posX, int posY, float fontSize, Color color) {
```

```
    if (!text || text[0] == '\0') return;
    Vector2 pos = { (float)posX, (float)posY };
    Font fontToUse = fontLoaded ? mainFont : GetFontDefault();
    DrawTextEx(fontToUse, text, pos, fontSize, 1.0f, color);
}
int MeasureAppText(const char* text, float fontSize) {
    if (!text || text[0] == '\0') return 0;
    Font fontToUse = fontLoaded ? mainFont : GetFontDefault();
    Vector2 sz = MeasureTextEx(fontToUse, text, fontSize, 1.0f);
    return (int)sz.x;
}
```

2. Custom Executive Button Component

```
// Custom Executive Button with hover state and crisp text centering
bool DrawExecutiveButton(Rectangle rect, const char* text, Color bgNormal, Color
bgHover, Color textCol, Color borderCol) {
    Vector2 mousePos = GetMousePosition();
    bool hovered = CheckCollisionPointRec(mousePos, rect);
    Color bg = hovered ? bgHover : bgNormal;

    DrawRectangleRounded(rect, 0.22f, 6, bg);
    DrawRectangleRoundedLinesEx(rect, 0.22f, 6, 1.2f, hovered ? textCol : Fade(borderCol,
0.8f));

    float fontSize = 15.0f;
    int textWidth = MeasureAppText(text, fontSize);
    DrawAppText(text, (int)(rect.x + (rect.width - textWidth) / 2.0f), (int)(rect.y + (rect.height -
fontSize) / 2.0f), fontSize, textCol);

    return hovered && IsMouseButtonPressed(MOUSE_BUTTON_LEFT);
}
```

3. Custom Input Field & Keyboard Handling

```
typedef struct {
    Rectangle rect;
    char text[128];
    int maxLen;
    bool active;
    const char* label;
    const char* placeholder;
} CustomInputBox;
// Input handling (typing, backspace, and focus toggle)
void UpdateCustomInput(CustomInputBox* box) {
    if (IsMouseButtonPressed(MOUSE_BUTTON_LEFT)) {
        box->active = CheckCollisionPointRec(GetMousePosition(), box->rect);
    }

    if (box->active) {
        int key = GetCharPressed();
        while (key > 0) {
            if ((key >= 32) && (key <= 125) && ((int)strlen(box->text) < box->maxLen)) {
                int len = (int)strlen(box->text);
                box->text[len] = (char)key;
                box->text[len + 1] = '\0';
            }
            key = GetCharPressed();
        }

        if (IsKeyPressed(KEY_BACKSPACE)) {
            int len = (int)strlen(box->text);
            if (len > 0) {
                box->text[len - 1] = '\0';
            }
        }
    }
}
// Rendering input field with blinking cursor
```

```
void DrawCustomInput(CustomInputBox* box, Color textColor, Color labelColor, Color bgBoxColor, Color borderColor, Color activeBorderColor) {
    if (box->label) {
        DrawAppText(box->label, (int)box->rect.x, (int)box->rect.y - 19, 14.0f, labelColor);
    }

    Color border = box->active ? activeBorderColor : borderColor;

    DrawRectangleRounded(box->rect, 0.18f, 4, bgBoxColor);
    DrawRectangleRoundedLinesEx(box->rect, 0.18f, 4, 1.4f, border);

    if (strlen(box->text) > 0) {
        DrawAppText(box->text, (int)box->rect.x + 8, (int)box->rect.y + 8, 14.0f, textColor);
    } else if (box->placeholder) {
        DrawAppText(box->placeholder, (int)box->rect.x + 8, (int)box->rect.y + 8, 14.0f, labelColor);
    }

    // Blinking cursor calculation
    if (box->active && (((int)(GetTime() * 2.2)) % 2 == 0)) {
        int textW = MeasureAppText(box->text, 14.0f);
        DrawRectangle((int)box->rect.x + 10 + textW, (int)box->rect.y + 8, 2, 18, activeBorderColor);
    }
}
```

4. Interactive Calendar Picker Modal Overlay

```
void DrawCalendarPickerModal(CalendarPicker* picker, Color currentPanel, Color currentText, Color currentMuted, Color currentAccent, Color currentHeader, Color currentBorder, Color currentRed) {
    if (!picker->active || !picker->targetInput) return;

    float calW = 270.0f, calH = 300.0f;
    float calX = picker->anchorPos.x, calY = picker->anchorPos.y;
    Rectangle calRect = { calX, calY, calW, calH };

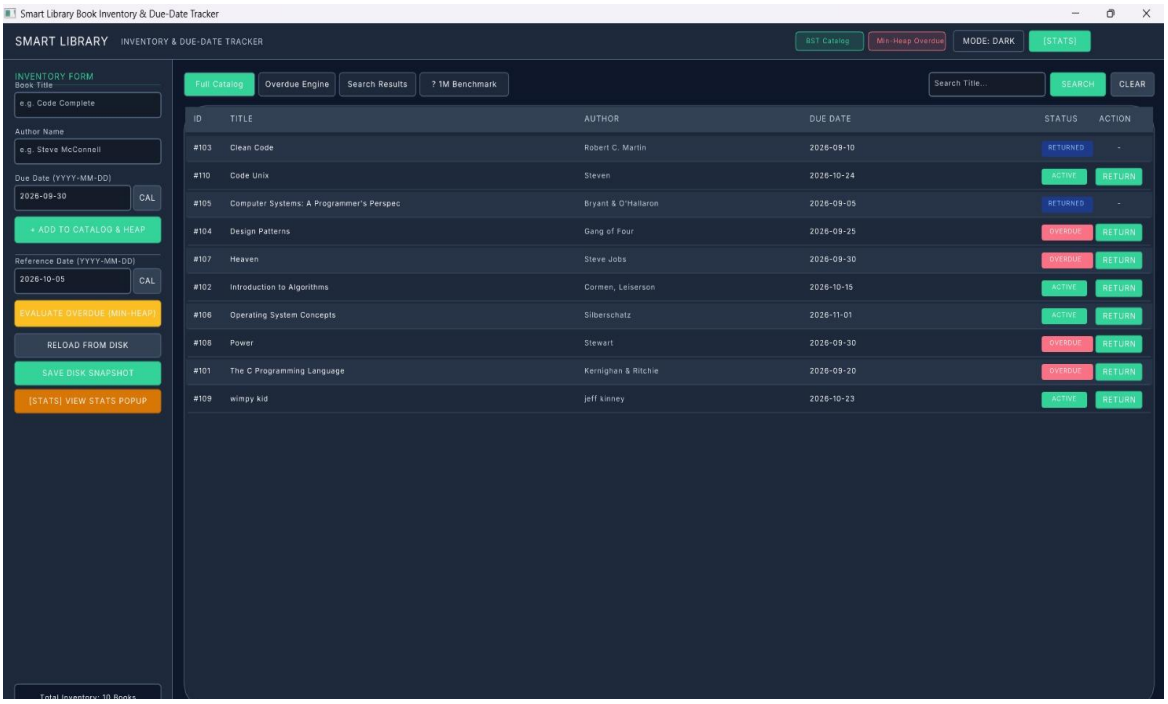
    // Shadow & Background
    DrawRectangleRounded((Rectangle){ calX + 4, calY + 4, calW, calH }, 0.12f, 4, Fade(BLACK, 0.35f));
    DrawRectangleRounded(calRect, 0.12f, 4, currentPanel);
    DrawRectangleRoundedLinesEx(calRect, 0.12f, 4, 1.5f, currentAccent);

    // Month/Year Header & Nav Controls ('<', '>', 'X')
    DrawRectangleRounded((Rectangle){ calX + 1, calY + 1, calW - 2, 40 }, 0.12f, 4, currentHeader);

    // Grid rendering for days of month (Mo-Su)
    int totalDays = getDaysInMonth(picker->viewYear, picker->viewMonth);
    int firstDow = getFirstDayOfWeek(picker->viewYear, picker->viewMonth);

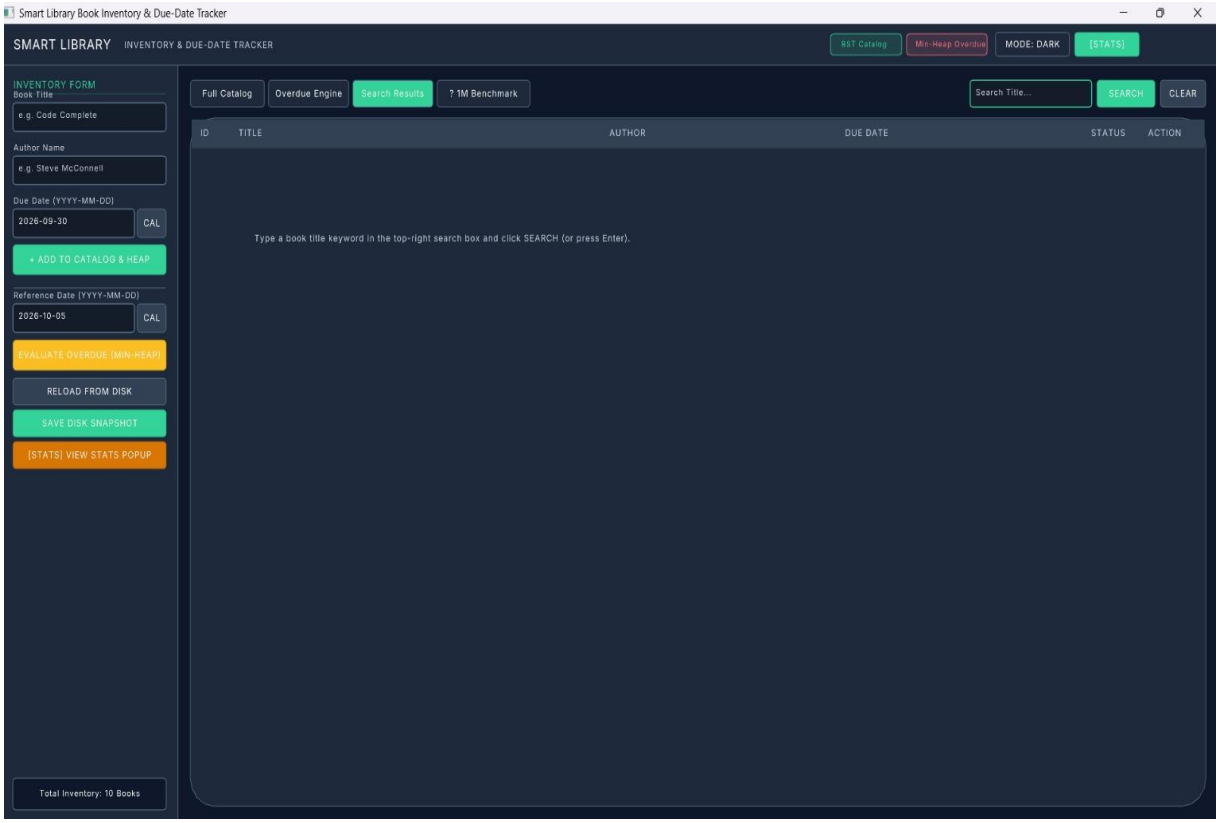
    // Render 6-row calendar date cell matrix with click listeners
    // The ui_manager.h header file defines the architectural state and layout contracts for the Raylib/Raygui desktop interface. It encapsulates active navigation tabs, form input buffers, theme color palettes, and modular rendering function prototypes. Together, these structures manage user interactions and visual states across the entire dashboard.
}
```

UI LAYOUT



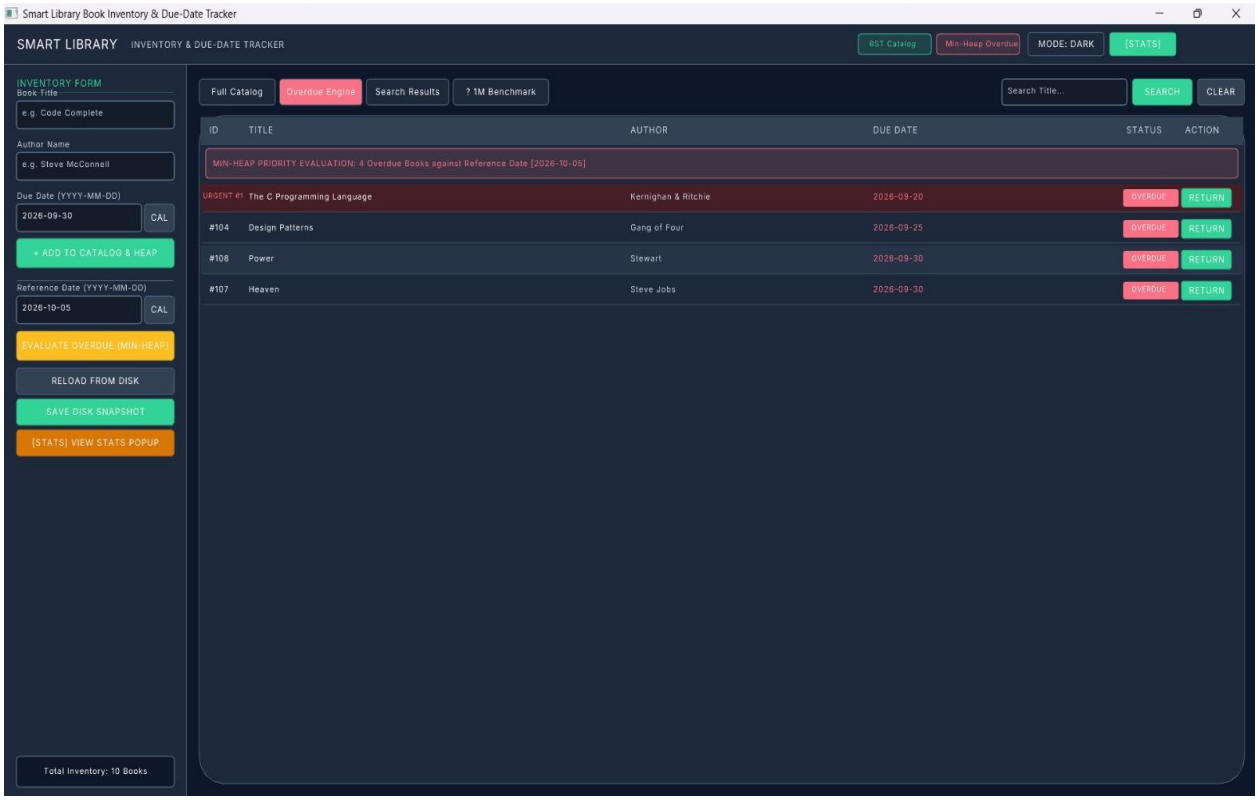
Catalogue Page of the App

The Catalogue page serves as the primary inventory dashboard, displaying all library book records organized alphabetically for rapid navigation. Powered by an underlying Binary Search Tree, it delivers logarithmic search efficiency alongside smart case-insensitive keyword filtering. Administrators can effortlessly monitor book details, inspect active or available states, and process immediate return actions directly from this view.



Search Engine of the App

The Search Engine module powers case-insensitive substring querying across the entire library catalogue, allowing users to type partial keywords to instantly surface relevant matches. Leveraging recursive BST traversal, it dynamically filters and populates the search results grid in real time. This ensures flexible and efficient book discovery without requiring strict exact-title inputs



Over-due Engine of the App

The Overdue Engine automatically flags unreturned books whose return deadlines have passed the current system reference date. Powered by a Min-Heap priority queue, this engine is essential for eliminating tedious manual date auditing by providing instant, constant-time access to the most critical overdue materials. This automated tracking safeguards inventory compliance and preventing the unmanaged book losses.

Conclusion

- **BST Catalogue Engine (bst.c & bst.h):** Ultimately, integrating this robust indexing mechanism transforms backend data management into a seamless, high-performance experience. By combining rapid algorithmic efficiency with flexible query tools, the system ensures that expanding library collections remain effortlessly navigable, future-proofing the inventory for both administrators and patrons alike.
- **Min-Heap Priority Queue Module (heap.c & heap.h):** Together, these capabilities shift overdue management from a reactive chore into a streamlined, proactive workflow. By automating chronological oversight, the priority framework eliminates administrative bottlenecks, ensuring time-sensitive library protocols execute with absolute precision.
- **Persistence & File I/O Engine (persistence.c):** Hence, this self-contained storage mechanism bridges the gap between volatile runtime memory and long-term durability. By eliminating external dependencies while safeguarding data integrity across sessions, the architecture achieves a lightweight, highly resilient foundation for seamless standalone deployment.
- **UI & Main Controller Module (main.c):** Therefore, by merging fluid, high-performance visual feedback with intuitive desktop layouts, this frontend layer completes a cohesive, human-cantered ecosystem. It bridges low-level data structures with an engaging, distraction-free interface, ensuring that complex library operations feel completely effortless for the end user.